Sistemas Distribuidos

Proyecto – Implementación de referencia

Diego Sevilla Ruiz

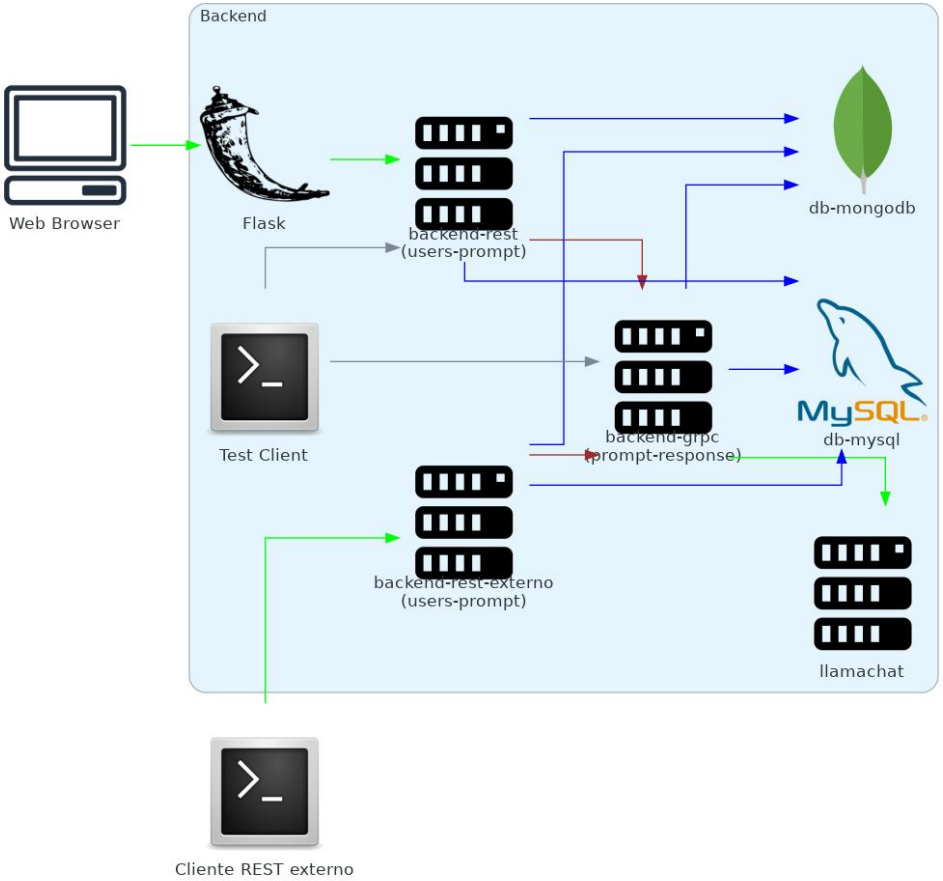
DITEC
Facultad de Informática

Murcia, 2026

Índice

1. Subproyectos y *rationale*
2. Subproyectos
3. Sistema de build

Arquitectura de la aplicación



Sistemas Distribuidos 2023-24

Subproyectos y *rationale*

- La implementación de referencia se ha realizado separando en varios subproyectos para compartir código
- La compartición se hace a través de `jars` creados con maven e instalados en el repositorio local (`$HOME/.m2` al ejecutar `mvn install`)
- Los proyectos del DAO y de los bindings gRPC se comparten
- La configuración permite la compartición a través de Eclipse y también a través de maven, así que todos los comandos maven funcionan correctamente
  - `es.um.sisdist.backend.dao`, y
  - `es.um.sisdist.backend.grpc.GrpcService`

Subproyectos y *rationale* II

- Estructura de directorios:

```
impl
|-- frontend
|    |-- app
|    `-- devel
|-- backend
|    `-- es.um.sisdist.backend.dao
|-- backend-grpc
|    |-- es.um.sisdist.backend.grpc.GrpcService
|    `-- es.um.sisdist.backend.grpc.GrpcServiceImpl
|-- backend-rest
|    `-- es.um.sisdist.backend.Service
|-- llamachat
|-- db-mongo
`-- db-mysql
```

Frontend

- El frontend es similar al ejemplo Flask
- Actualmente no está conectado con el backend
- Hay que conectarlo e implementar las vistas y el envío de peticiones REST

Backend

- El directorio `backend` contiene el proyecto del DAO `es.um.sisdist.backend.dao`
- Se ha hecho una implementación de DAO, **revisadla y cambiadla como sea necesario**
- Se ha implementado el DAO de usuarios, pero no el del resto de elementos de la implementación
- Se aportan implementaciones para MySQL y para MongoDB
- El DAO se conecta al servidor de bases de datos usando la variable de entrono `SQL_SERVER`/`MONGO_SERVER`, y si no están definidas, "`localhost`"
- Esto se puede ver en el fichero de build `docker-compose.yml`
- Esto permite hacer pruebas sin lanzar todos los contenedores
- El subproyecto se puede instalar con `mvn install`, y es usado por el resto de subproyectos que acceden a la BD
- En Eclipse se han incluido en el *Build Path*

Backend gRPC

- Está compuesto por los dos proyectos `GrpcService` y `GrpcServiceImpl`
- El proyecto `GrpcService` contiene los *stubs* y *skeletons* para gRPC
- Será compartido por los proyectos que usan gRPC (tanto cliente como servidor)
- Se ha definido el fichero `grpcservice.proto` **estudiadlo y ver cómo modificarlo**
- También se deberán instalar antes con `mvn install`
- El proto contiene el interfaz:

Backend gRPC II

```
syntax = "proto3";

service GrpcService
{
    rpc Ping(PingRequest) returns (PingResponse);
}

message PingRequest
{
        int32 v = 1;
}

message PingResponse
{
        int32 v = 1;
}
```

- El proyecto `GrpcServiceImpl` implementa el servidor gRPC, y utiliza el DAO
- El servidor funciona, pero sólo contiene la funcionalidad de Ping
- **Hay que implementar las funciones de gestión de las peticiones gRPC del proyecto en un *Thread***

Backend gRPC III

- El proyecto también incluye una serie de ejemplos y pruebas como ejemplo de implementación de gRPC

Backend-REST

- El directorio `backend-rest` contiene el subproyecto `es.um.sisdist.backend.Service`
- Es un proyecto REST con Tomcat 10.1
- Utiliza el DAO y el de los *bindings* gRPC
- El proyecto implementa los puntos de acceso REST especificados en el proyecto, y también el objeto *singleton* `AppLogicImpl`, que tiene métodos para realizar cada una de las funciones
- Esto permite separar el acceso REST de la implementación de la lógica de la aplicación
- Están implementados algunos puntos de acceso, pero no todos. Hay que implementarlos

db-mysql

- El directorio `db-mysql` implementa el contenedor MySQL para la base de datos
- Al construir el contenedor se le proporciona un esquema que podéis ver a continuación. Está conectado con los objetos DAO:

```sql
CREATE SCHEMA IF NOT EXISTS ssdd;
USE ssdd;

CREATE TABLE IF NOT EXISTS users(
        id varchar(50),
        email varchar(50),
        password_hash text,
        name text,
        TOKEN text,
        visits int,
        PRIMARY KEY(id)
);

-- Para búsquedas con email
CREATE INDEX user_email_idx ON users (email);

-- CUIDADO!! AÑADO UN USUARIO PARA PROBAR, PASSWORD: "admin"
INSERT INTO users VALUES ("ddd", "dsevilla@um.es", "21232
     f297a57a5a743894a0e4a801fc3", "diego", "TOKEN", 0);
```

db-mysql II

- Se inserta una tupla de usuarios para probar. Habrá que quitarla cuando la aplicación se ponga en producción

dsevilla:ssdd-llamachat:1.0

- Se ofrece el contenedor `dsevilla:ssdd-llamachat:1.0` como se vió en la especificación del proyecto
- Se puede usar ese, pero para acelerar el funcionamiento, se puede hacer uso de otro proyecto dummy
- Así que en los diferentes ficheros `docker-compose.yml` se puede cambiar:
  - `dsevilla:ssdd-llamachat:1.0`
  - por:
  - `dsevilla:ssdd-llamachat-dummy:1.0`

Build

- Cada subproyecto de los nombrados construye un contenedor Docker
- Para construirlos, existen ficheros `Dockerfile` dentro de los subdirectorios
- También hay dos ficheros `docker-compose.yml` (para producción) y `docker-compose-devel.yml` (para desarrollo)
- La diferencia es que el que no es de desarrollo utiliza imágenes finales, mientras que el *devel* reconstruye las imágenes cada vez que se lanza
- De esta forma se pueden aplicar los cambios mientras se desarrolla de forma más rápida
- También se ha desarrollado un `Makefile` que compila todos los subproyectos en el orden necesario
- Con el *target* `containers` (`make containers`) construye las imágenes de los contenedores

Build II

- El `Makefile` entra en los subdirectorios, donde también hay otro `Makefile`

Build III

- Pasos para la ejecución de la práctica:

```
$ cd proyecto
$ make
$ docker-compose -f docker-compose-devel.yml up
```

- El frontend se puede acceder en la dirección `http://localhost:5010`

Build IV

- Los ficheros `.yml` también ofrecen el puerto 8080 para acceder al servidor REST (no estará así en el despliegue final)
- Para probar el servicio REST se puede utilizar la URL `http://localhost:8080/Service`
- Para probar el login:

```
$ curl -v -XPOST -HContent-type:application/json -HAccept:application/json \
    localhost:8080/Service/checkLogin \
    --data '{"email": "dsevilla@um.es", "password": "admin"}'
```